

Verification Timing: Why Checking AI-Written Code at the Gate Is Already Too Late

Yunhao Jiao | TestSprite

About Me

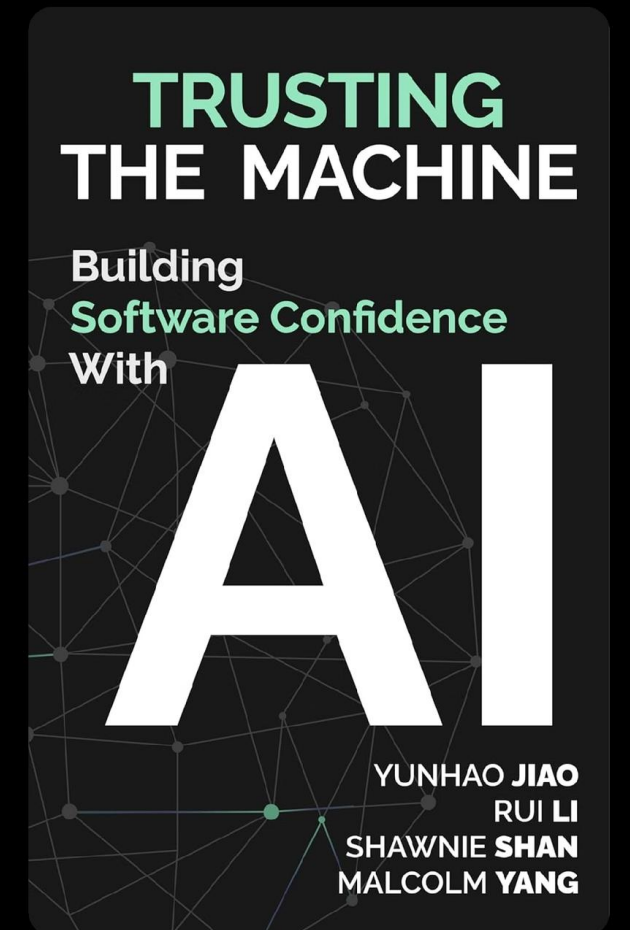
Experience

MS Yale

previous AWS Senior Software Engineer

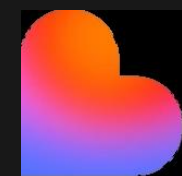
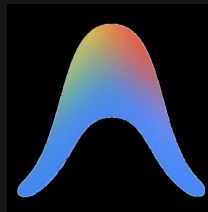
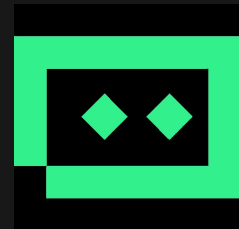
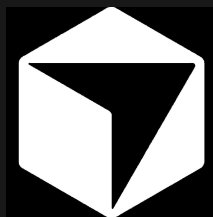
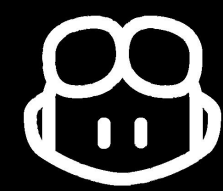
First AI paper (NLP) in 2017

Built Contract Tests framework for AWS::CFN





Tools like Claude Code, Cursor, Codex, Lovable, Replit, and etc. have made coding much easier than ever!

**Lovable**

But Vibe Coding Is Introducing New Pains

① The pain of overwhelming output

AI coding agents can run for hours and are able to generate **thousands of or even more lines of code**, making it nearly impossible for users to manually review or understand the results.

② The pain of AI-generated code inaccuracies

Right now, AI tools can produce code that compiles but doesn't function as intended **40% accuracy**). [Ref] Generating the first 60–70% of draft code with AI took **only 10 minutes**, but fixing the remaining 30% of mismatches or issues **took days**.

③ The pain of losing track of requirements (context window)

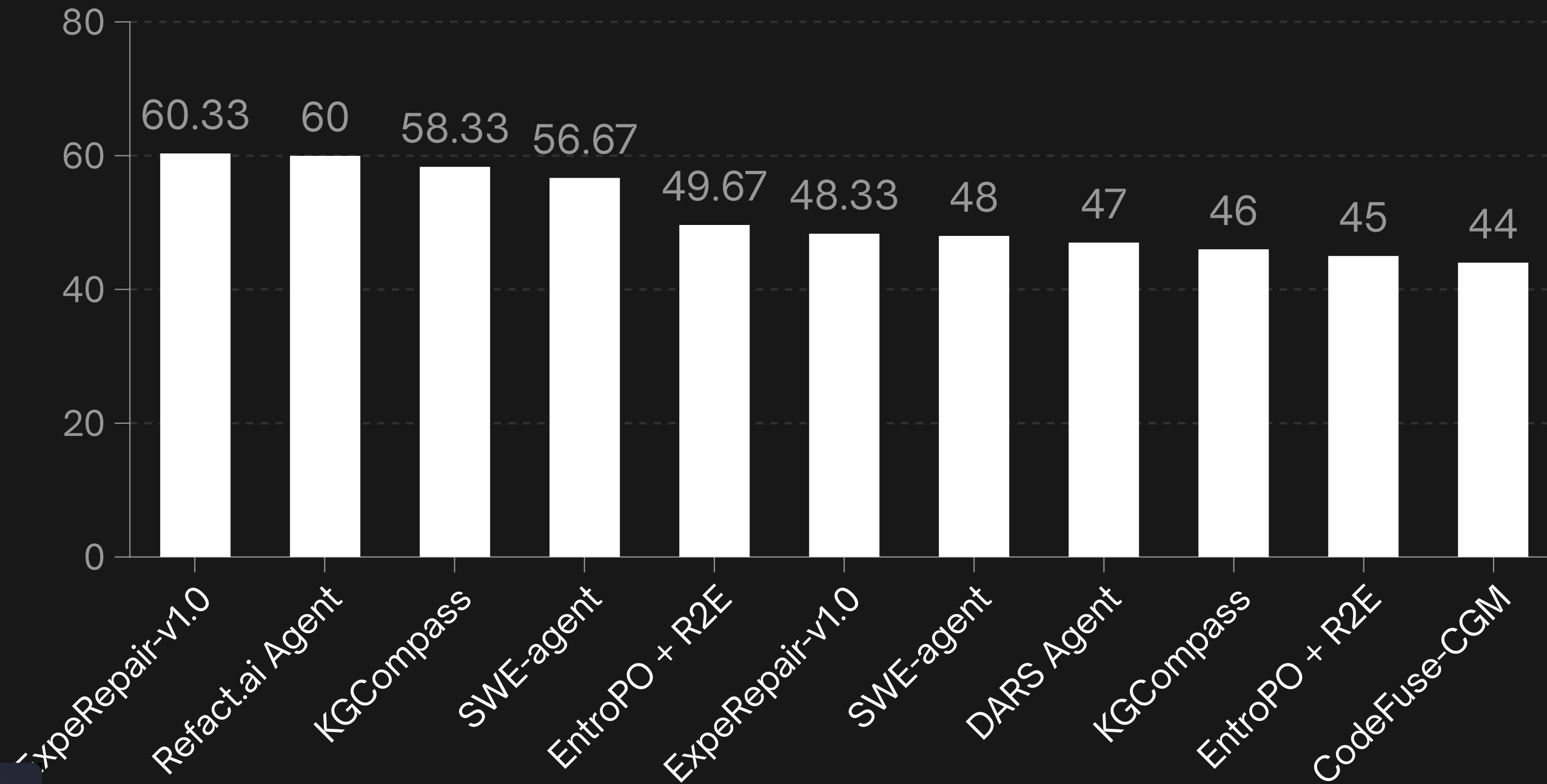
No context window holds every requirement across a **long build**. Constraints get dropped mid-task, and nothing brings them back unless someone reminds the agent.

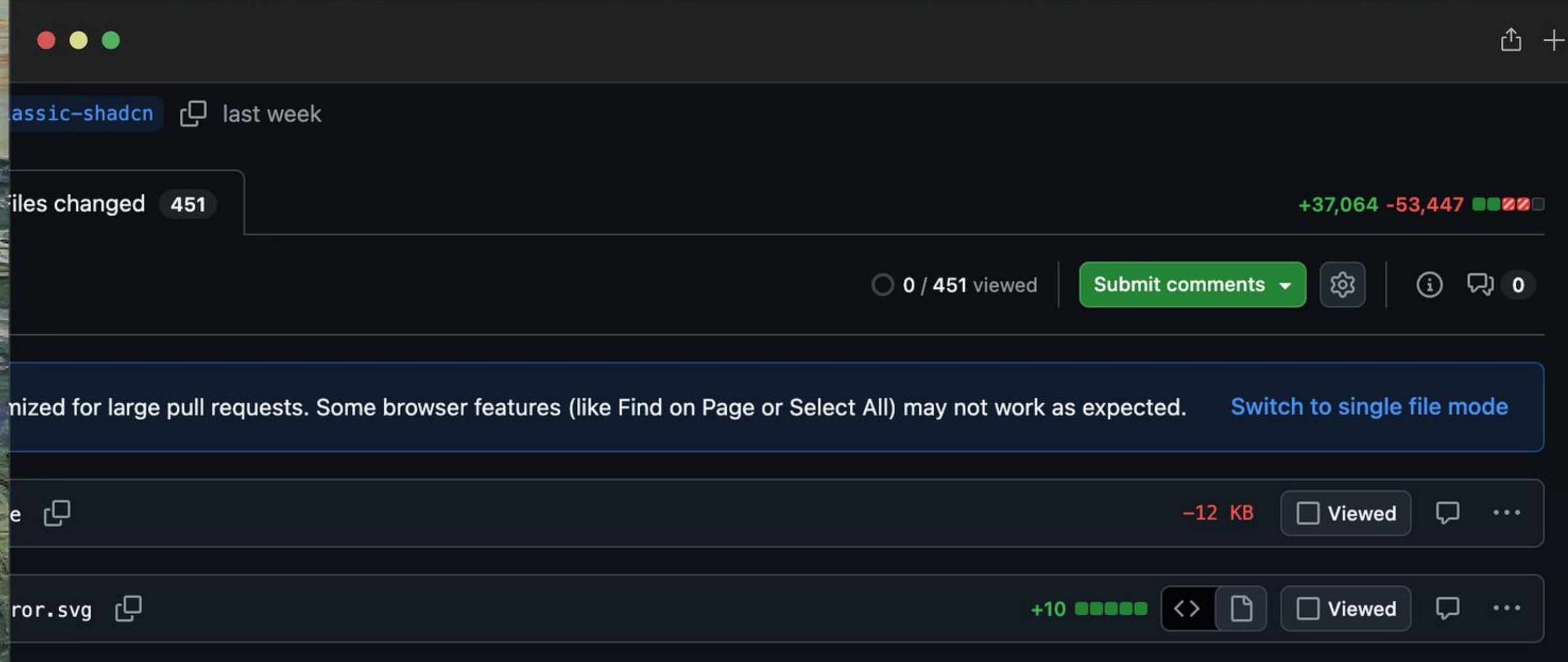


Problem

[Ref]

But Vibe Coding Is Introducing New Pains





AI: A Game-Changer in Coding

The PR is 30,000 lines long. Be honest — are you really reviewing it?



> Btw for the cleanup feature work, can you double check and are you sure you really did everything right?

- Good question – let me actually re-audit. The cleanup work spanned multiple commits over 2 sessions, and I made a few decisions worth double-checking. Let me verify each piece against the real data you pasted earlier.
- **Bash**(cd /Users/jiaoyunhao/Downloads/TestSprite-V2-Frontend && git log --oneline --all --author="Yunhao|Claude" -50 -- "src/components/charts/CleanupPanel/" "src/ser...")
L 3b2ada40 fix(cleanup): banner follows plan view so rows and totals agree e251f8a5 fix(cleanup): decode captured.value wire form so sweptIds comparison matches ad515812 fix: cleanup count parity, queued-state label, silent orphan pre-flight
... +16 lines (ctrl+o to expand)
L Shell cwd was reset to /Users/jiaoyunhao/Downloads/testsprite-langfuse-prompts

AI: A Game-Changer in Coding

AI verifying AI? AI code review? AI-written unit tests ? AI-run integration tests ?

The Requirements in Today's Software Testing

- ① AI code review: reads the code, doesn't run it.
- ① AI-written unit tests: tests reflect the code, not the spec(requirements).
- ① AI-run integration tests: catches it, if it runs.



Andrew Ng, co-founder of Google Brain, recently emphasized:



Agentic Coding and Agentic Software Testing Go Together: Agentic coding can make mistakes, but agentic testing can find and fix them.

Dear friends, Automated software testing is growing in importance in the era of AI-assisted coding. Agentic coding systems accelerate development but are also unreliable. Agentic testing — where you ask AI to write tests and check your code against them — is helping. Automatically testing infrastructure software components that you intend to build on top of is especially helpful and results in more stable infrastructure and less downstream debugging.

✓ (Integration) **TESTING** for **AI-generated code** is a MUST-HAVE.



The way we test software is evolving, and when we test it is changing too.

Verification is moving out of
the gate and into the
agent's own loop.

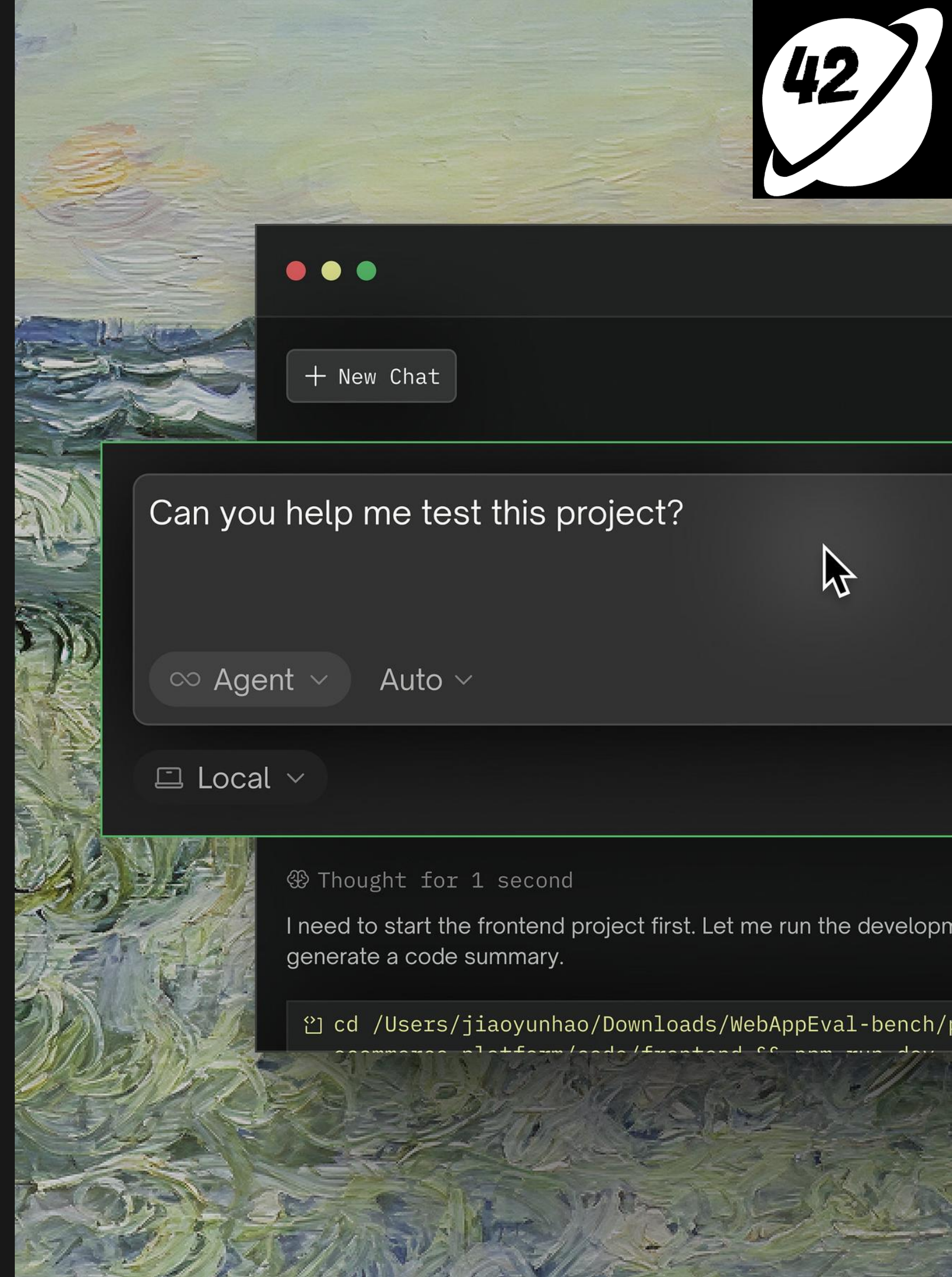


An experiment

An Experiment We Ran: CoderCup

We asked several **frontier agents** to build the same project from **the same requirements** and watched how it turned out. The results were very interesting.

Try Pitch



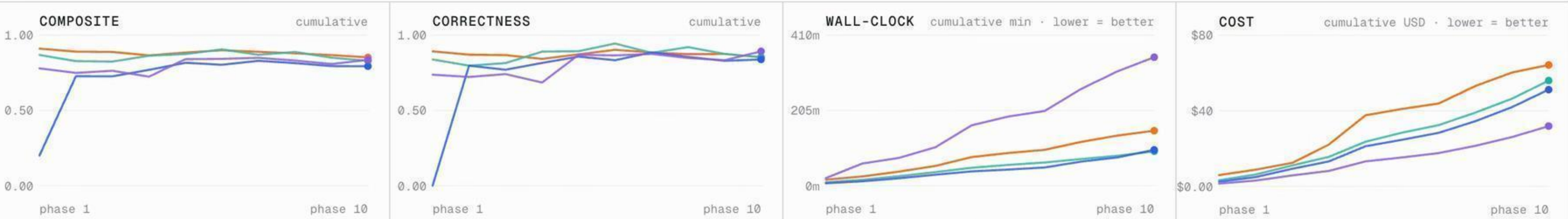
4 AGENTS · 10 OF 10 PHASES · LAST SCORED 2026-09-28

Claude Code 0.852 ★

Codex 0.829

Anti-Gravity 0.793

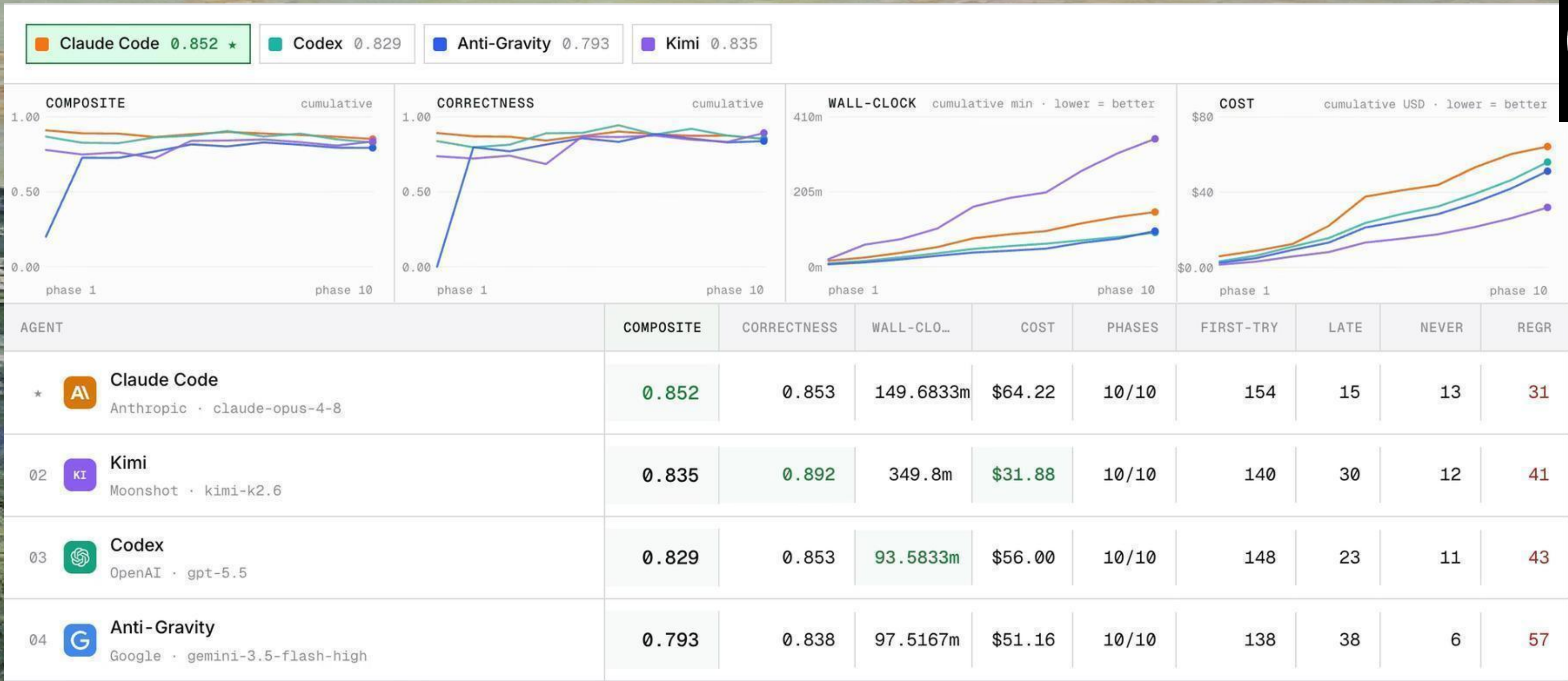
Kimi 0.835



AGENT		COMPOSITE	CORRECTNESS	WALL-CLO...	COST	PHASES	FIRST-TRY	LATE	NEVER	REGR
★	AI Claude Code Anthropic · claude-opus-4-8	0.852	0.853	149.6833m	\$64.22	10/10	154	15	13	31
02	KI Kimi Moonshot · kimi-k2.6	0.835	0.892	349.8m	\$31.88	10/10	140	30	12	41
03	Codex OpenAI · gpt-5.5	0.829	0.853	93.5833m	\$56.00	10/10	148	23	11	43
04	G Anti-Gravity Google · gemini-3.5-flash-high	0.793	0.838	97.5167m	\$51.16	10/10	138	38	6	57

Trajectory updates after each phase clears its TestSprite gate

[Download raw run data ↗](#)

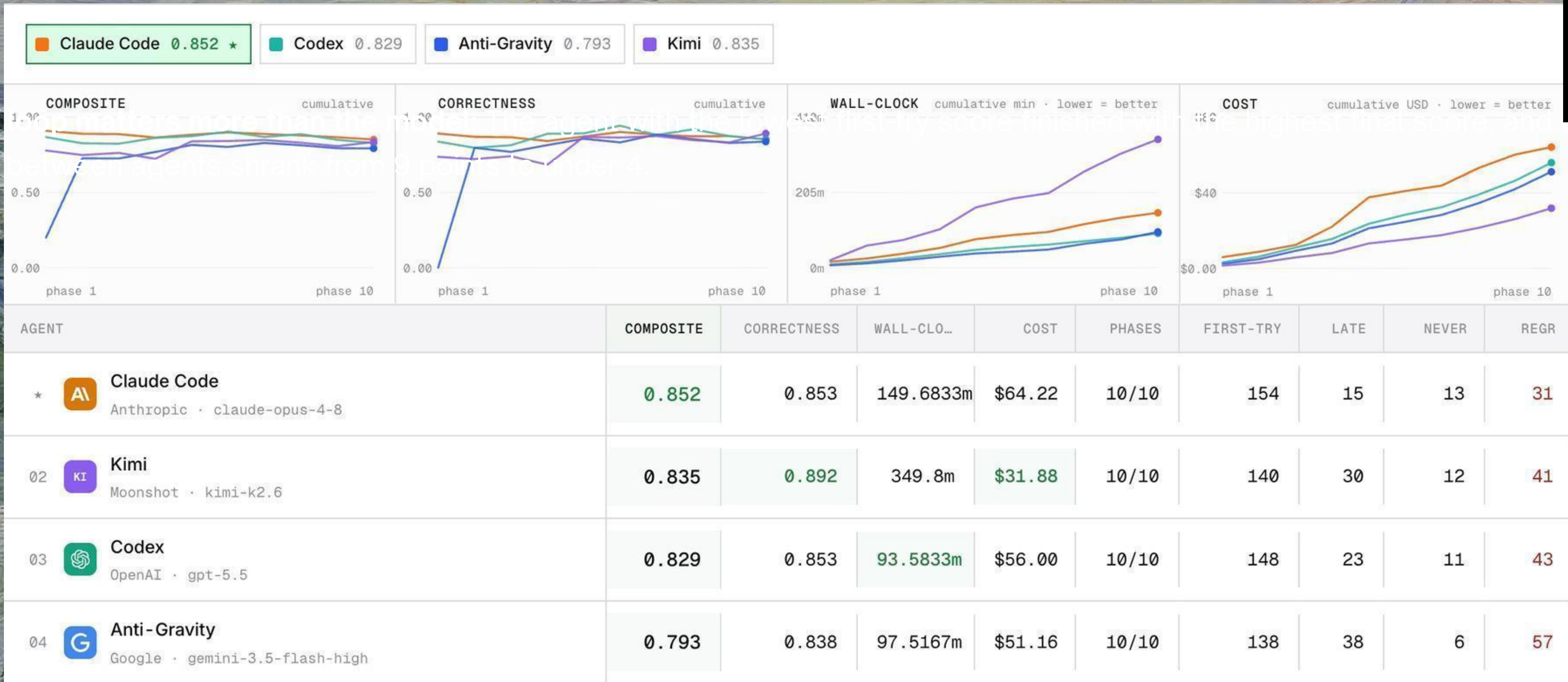


1

What CoderCup showed:

The loop lifts every agent: Agents got about 80% of tasks right on the first try, and 94% once verification caught the failure and drove the fix.

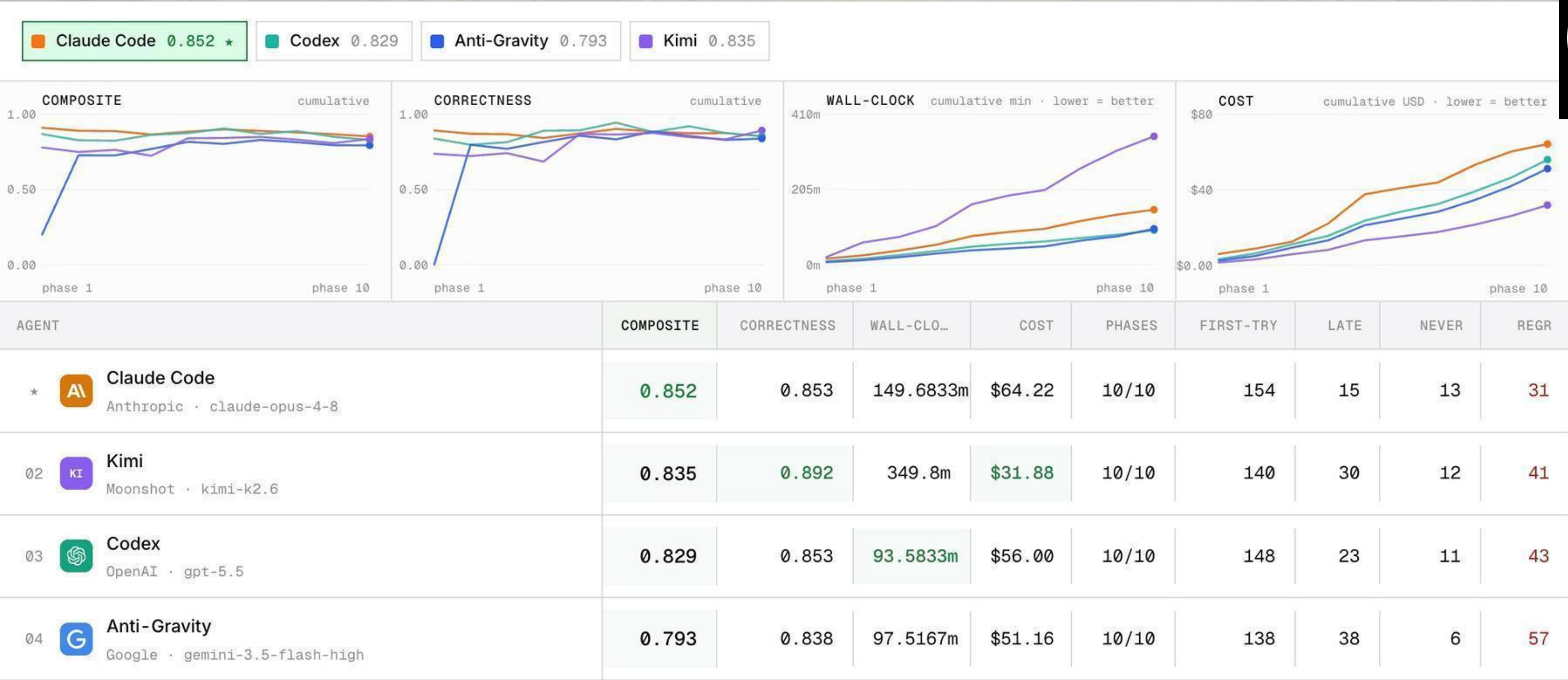
The gap



2

Today's AI testing agent can already:

The loop matters more than the model: The agent that finished with the **highest correctness** started near the **bottom on first-try** tasks, and it was also **the cheapest agent** in the field.



3

Today's AI testing agent can already:

Every agent breaks its own work: Each agent **broke features it had already gotten working** ~40 times in the loop, and every break stayed hidden until the app ran.

ct_Information.py — WebAppEval-bench

Displays_Correct_Information.py × #

03_Product_Catalog_Displays_Correct_In

Can you help me test this project?

∞ Agent ▾

Auto ▾



Thought for 7 seconds

I'll help you test this project using TestSprite. Let me first explore the project structure to understand what we're working with, and then set up and run the tests.

Thought for 12s and read ecommerce-platform and 5 other files



Today's AI testing agent can already:

Verify inside the loop: The coding agent calls the tests itself while it works, so it finds and fixes each break before moving on.

Modified At

Status

⚡ 7/17 Pass
2025-09-21 13:12

Type	Latest Status	Modified At
Frontend	✓ Pass	2025-09-21 13:16

Share Feedback

M

17 tests selected Cancel **Deploy Tests**



Today's AI testing agent can already:

Keep every test it verifies: Each passing test joins a suite that reruns on every change, from the agent's loop to the pull request to production.

How does this solves the existing verification problem?

- ✓ Massive time savings (run in parallel on cloud ~10 minutes)
- ✓ Fully automated workflow
- ✓ Cost-effective testing solutions (<\$50 a month)
- ✓ Less coding/QA knowledge required

Integration?

During development

CLI/MCP + Github Actions

Post release

Cloud-based management





Thank You!

Every company should have
high-quality software!